Mandatory Activity 4

Report, group JanAlpNiels

R1 (Spec)
We have built a Ricart-Agrawala distributed mutual exclusion in Go with gRPC. The critical section is represented by a log saying: “Node [X] is accessing the critical section”. Every node is both a server and a client. By pressing Enter in any node at any time, the node requests access to the Critical Section. In the Critical Section there is a 10 second sleep before releasing it. Peer discovery uses three hardcoded localhost ports: 50051–50053, assigned to unique node IDs 1–3.

R2 (Safety)
Our code is written in a way that makes sure that only one node enters the Critical Section at a time. Only after having received access granted by all the other nodes the requesting node enters the Critical Section.

A node that is “held”, or “wanted” with a lower (timestamp, ID) pair than the requester, withholds its reply until it has left the CS and entered “released”. So if two nodes request at the same time, the one with the lower pair is answered first, and the other must wait until it releases. The lower timestamp wins and if it's a tie the lower nodeID wins.

Exclusion comes from the replies, not from a lock. The mutex we are using protects the data of the node from being accessed by other goroutines when data is read or changed. This prevents a data race on the state for example. We ran the system with Go’s race detector (go run -race), which reported no data races.

R3 (Liveliness)
A request that is not granted immediately is put in a queue, and the node answers every queued request when it leaves the Critical Section. Requests are totally ordered by (timestamp, ID), so among all waiting nodes there is always one with the smallest pair. That node is not deferred by any other waiting node, and a node inside the Critical Section answers it when it leaves, so it receives all its replies and enters. A node in the Critical Section always leaves it (after 10 seconds), and then answers everyone it deferred. Every request eventually gains access provided all peers remain available and permission RPCs complete successfully within their 60 second deadlines.

The algorithm assumes that no node crashes. Since every node must reply, no node can enter the Critical Section while one is down. In our implementation, a request to an unresponsive node fails (immediately, or after a 60-second timeout), and the requesting node gives up and returns to the released state. This could be mitigated by removing a node from the list of peers once it has been decided to be unresponsive.

Log example from Node2's point of view

Node2:

16:03:36 Enter pressed, sending access request	***N2 asking for request***
16:03:36 Node 2: REQUESTING CS, timestamp=4	***req with t=4 created***
16:03:36 Node 2 sending REQUEST to Node 3	***req sent to N3***
16:03:36 Node 2 sending REQUEST to Node 1	***req sent to N1***

Node3:

16:03:36 Node 3 received request from node 2 with timestamp 4	***N3 received req***
16:03:36 Node 3 is in released state, granting access to node 2	***gives access to N2***

Node1:

16:03:36 Node 1 received request from node 2 with timestamp 4	***N1 receives req***
16:03:36 Node 1 is in held state, cannot grant access to node 2	***N1 is held, no access given***

Node2:

16:03:36 Node 2 received REPLY from Node 3	***N2 receives access from N3***

Node3:

16:03:37 Node 3: REQUESTING CS, timestamp=6	***N3 creates req with t=6***
16:03:37 Node 3 sending REQUEST to Node 2	***N3 sends a req to N2***

Node2:

16:03:37 Node 2 received request from node 3 with timestamp 6	***N2 receives req from N3***
16:03:37 Node 2 is in wanted state, cannot grant access to node 3	***N2 is wanted, no access given***

Node1:

16:03:43 Critical section is released by node 1	***N1 is releasing the CS***
16:03:43 Node 1 granting deferred permission to Node 2	***N1 gives access to N2***

Node2:

16:03:43 Node 2 received REPLY from Node 1	***N2 receives access from N1***
16:03:43 Node 2 is accessing the critical section	***N2 accesses CS***
16:03:53 Critical section is released by node 2	***N2 releases CS***
16:03:53 Node 2 granting deferred permission to Node 3	***N2 gives deferred access to N3***

Link to the repository

https://github.com/alqeren1/Distsystems/tree/main/Ass4_Jan

Appendix

Logs

NODE1

```
Ass4_Jan % go run -race . -nodeID 1 -port :50051
2026/11/08 16:03:20 released
2026/11/08 16:03:20 Running Node
2026/11/08 16:03:20 clients created: 2
2026/11/08 16:03:33 Enter pressed, sending access request
2026/11/08 16:03:33 Node 1: REQUESTING CS, timestamp=2
2026/11/08 16:03:33 Node 1 sending REQUEST to Node 3
2026/11/08 16:03:33 Node 1 sending REQUEST to Node 2
2026/11/08 16:03:33 Node 1 received REPLY from Node 2
```

2026/11/08 16:03:33 Node 1 received REPLY from Node 3
2026/11/08 16:03:33 Node 1 is accessing the critical section
2026/11/08 16:03:36 Node 1 received request from node 2 with timestamp 4
2026/11/08 16:03:36 Node 1 is in held state, cannot grant access to node 2
2026/11/08 16:03:37 Node 1 received request from node 3 with timestamp 6
2026/11/08 16:03:37 Node 1 is in held state, cannot grant access to node 3
2026/11/08 16:03:43 Critical section is released by node 1
2026/11/08 16:03:43 Node 1 granting deferred permission to Node 2
2026/11/08 16:03:43 Node 1 granting deferred permission to Node 3

NODE2

Ass4_Jan % go run -race . -nodeID 2 -port :50052
2026/11/08 16:03:22 released
2026/11/08 16:03:22 Running Node
2026/11/08 16:03:22 clients created: 2
2026/11/08 16:03:33 Node 2 received request from node 1 with timestamp 2
2026/11/08 16:03:33 Node 2 is in released state, granting access to node 1
2026/11/08 16:03:36 Enter pressed, sending access request
2026/11/08 16:03:36 Node 2: REQUESTING CS, timestamp=4
2026/11/08 16:03:36 Node 2 sending REQUEST to Node 3
2026/11/08 16:03:36 Node 2 sending REQUEST to Node 1
2026/11/08 16:03:36 Node 2 received REPLY from Node 3
2026/11/08 16:03:37 Node 2 received request from node 3 with timestamp 6
2026/11/08 16:03:37 Node 2 is in wanted state, cannot grant access to node 3
2026/11/08 16:03:43 Node 2 received REPLY from Node 1
2026/11/08 16:03:43 Node 2 is accessing the critical section
2026/11/08 16:03:53 Critical section is released by node 2
2026/11/08 16:03:53 Node 2 granting deferred permission to Node 3

NODE3

Ass4_Jan % go run -race . -nodeID 3 -port :50053
2026/11/08 16:03:24 released
2026/11/08 16:03:24 Running Node
2026/11/08 16:03:24 clients created: 2
2026/11/08 16:03:33 Node 3 received request from node 1 with timestamp 2
2026/11/08 16:03:33 Node 3 is in released state, granting access to node 1
2026/11/08 16:03:36 Node 3 received request from node 2 with timestamp 4
2026/11/08 16:03:36 Node 3 is in released state, granting access to node 2
2026/11/08 16:03:37 Enter pressed, sending access request
2026/11/08 16:03:37 Node 3: REQUESTING CS, timestamp=6
2026/11/08 16:03:37 Node 3 sending REQUEST to Node 2
2026/11/08 16:03:37 Node 3 sending REQUEST to Node 1
2026/11/08 16:03:43 Node 3 received REPLY from Node 1
2026/11/08 16:03:53 Node 3 received REPLY from Node 2
2026/11/08 16:03:53 Node 3 is accessing the critical section
2026/11/08 16:04:03 Critical section is released by node 3